DATA 260 - Homework 9: Redis View Counter, Semantic Response Cache and a Self-Improving Two-Agent System

GitHub repository: https://github.com/pragnayap/data260-3170

Section 0 - Configuration

Value	Definition	Mine
SID4	last 4 of SJSU student ID	3170
PORT_BASE	8000 + (SID4 mod 900)	8470
PREFIX	"s" + SID4	s3170
SEED	SID4	3170
VERIFY_SEED	260000 + SID4	263170
DOMAIN_ID	SID4 mod 8	2 - Municipal Transit Incidents (entity: Transit Incident)
Hardware		Intel Core i9-9880H (x86_64), 16 GB RAM, macOS 15, Docker Desktop
Models and services		Cache: Redis Stack 7.4 (RediSearch HNSW, COSINE, 384-d) in Docker, key prefix s3170:. Embeddings: all-MiniLM-L6-v2. Agents: qwen3:8b via Ollama, CPU-only. Source of truth: MySQL s3170_rel
Tagged commit hash	tag hw9	

Overview - what I built

Part	What I built	Result
1	INCR-based view counter for the primary entity (Transit Incident) in Redis, written to MySQL incidents.view_count every 10th view; also wired into GET /api/incidents/{id}.	25 accesses: sync after #10 and #20; Redis 25, MySQL 20
2	Semantic response cache at the ingress of the HW8 agent path: Redis vector index, cosine similarity, threshold 0.85, logs is_cached, similarity and latency.	14 queries: hit rate 57.1%; 538 ms cached vs 30,959 ms not cached
3	40-case labeled set (committed before the sweep), threshold sweep 0.75–0.95, one false hit in full, cost model, and a stale-answer demo with version-marker invalidation.	Ship 0.95 (cheapest, not good); stale answer caught and evicted
4	Self-improving system: Answering Agent, rule-based Evaluator Agent (max 2 attempts), lessons.json, validation gate, four variants.	Baseline 10/14; Repair, Lesson Memory, Validated Improvement 14/14; lesson_001 kept, lesson_002 rejected
-	Self-check (make verify-hw09).	22/22 checks pass

Part 1 - Redis View Counter

What I did

- Primary entity: Transit Incident. The counter key is s3170:views:incident:<id>, so every Redis key carries the s3170 prefix.
- One new method, increment_incident_views(incident_id), in the database/cache class IncidentCache (code/web_application/cache_db.py). It runs INCR, and when the count is a multiple of 10 it writes the count to incidents.view_count in MySQL and returns the count.
- The MySQL write is a raw UPDATE on purpose. An ORM update would change updated_at, but a view is not a content change, and updated_at is the data-version marker the semantic cache uses in Part 3.5.
- GET /api/incidents/{id} calls the method and returns the count in an X-View-Count header. If Redis is unavailable the read still succeeds.
- Because the sync happens only every 10th view, MySQL can lag Redis by up to 9 views. That is the design, not a bug.

Code

```
    def increment_incident_views(self, incident_id: int) -> int:
        """INCR the Redis counter; on every 10th view write it to incidents.view_count; return the count."""
        count = int(self.redis_client.incr(self.views_key(incident_id)))
        if count % SYNC_EVERY == 0:
            with self.session_factory() as db:
                # Raw UPDATE on purpose: the ORM would fire updated_at's onupdate, and a view is not
                # a content change (updated_at is the data-version marker used by the HW9 semantic cache).
                db.execute(text("UPDATE incidents SET view_count = :c WHERE id = :i"),
                           {"c": count, "i": incident_id})
                db.commit()
            log.info("Synced views to database (incident %s = %s)", incident_id, count)
            self.last_synced = count
        return count
```

Evidence

```
Pragnaya Priyadarshini HW9 - part1_counter.py
key = s3170:views:incident:1

=== Testing Counter ===

Access #1: Total = 1
Access #2: Total = 2
Access #3: Total = 3
Access #4: Total = 4
Access #5: Total = 5
Access #6: Total = 6
Access #7: Total = 7
Access #8: Total = 8
Access #9: Total = 9
Access #10: Total = 10
✓ Synced views to database
Access #11: Total = 11
Access #12: Total = 12
Access #13: Total = 13
Access #14: Total = 14
Access #15: Total = 15
Access #16: Total = 16
Access #17: Total = 17
Access #18: Total = 18
Access #19: Total = 19
Access #20: Total = 20
✓ Synced views to database
Access #21: Total = 21
Access #22: Total = 22
Access #23: Total = 23
Access #24: Total = 24
Access #25: Total = 25

Redis s3170:views:incident:1 = 25
MySQL incidents.view_count (id=1) = 20   (last synced at the 20th access)
```

```
❯       make counter-hw09
cd code/hw09 && ../../.venv/bin/python -u part1_counter.py
Pragnaya Priyadarshini HW9 - part1_counter.py
key = s3170:views:incident:1

=== Testing Counter ===

Access #1: Total = 1
Access #2: Total = 2
Access #3: Total = 3
Access #4: Total = 4
Access #5: Total = 5
Access #6: Total = 6
Access #7: Total = 7
Access #8: Total = 8
Access #9: Total = 9
Access #10: Total = 10
✓ Synced views to database
Access #11: Total = 11
Access #12: Total = 12
Access #13: Total = 13
Access #14: Total = 14
Access #15: Total = 15
Access #16: Total = 16
Access #17: Total = 17
Access #18: Total = 18
Access #19: Total = 19
Access #20: Total = 20
✓ Synced views to database
Access #21: Total = 21
Access #22: Total = 22
Access #23: Total = 23
Access #24: Total = 24
Access #25: Total = 25

Redis s3170:views:incident:1 = 25
MySQL incidents.view_count (id=1) = 20   (last synced at the 20th access)
(base)
```

```
data260-3170 on  main [✘!?⇡] via 🐍 v3.13.5 on ☁️  (us-east-1)
❯ docker exec s3170_redis redis-cli get s3170:views:incident:1
25
(base)
data260-3170 on  main [✘!?⇡] via 🐍 v3.13.5 on ☁️  (us-east-1)
❯
```

Part 2 - Semantic Response Cache

What I did

- The cache sits in front of the HW8 agent path. Each query is embedded with all-MiniLM-L6-v2 (384-d, normalised) and a KNN-1 search runs against a Redis HNSW vector index using the COSINE metric. Similarity = 1 − cosine distance.
- If the nearest cached query scores at or above the threshold (0.85) the stored answer is returned and the agent is not run (CACHE HIT). Otherwise the planner → DB-fact lookup → writer → reviewer path runs and the answer is stored (CACHE MISS).
- The agent path runs in-process rather than over Kafka, and it reads s3170_rel rows, so its answers can go stale (Part 3.5).
- Every request logs is_cached, similarity and latency. 14 test queries were used: new questions, exact duplicates, paraphrases, a near miss and an unrelated question.

Code

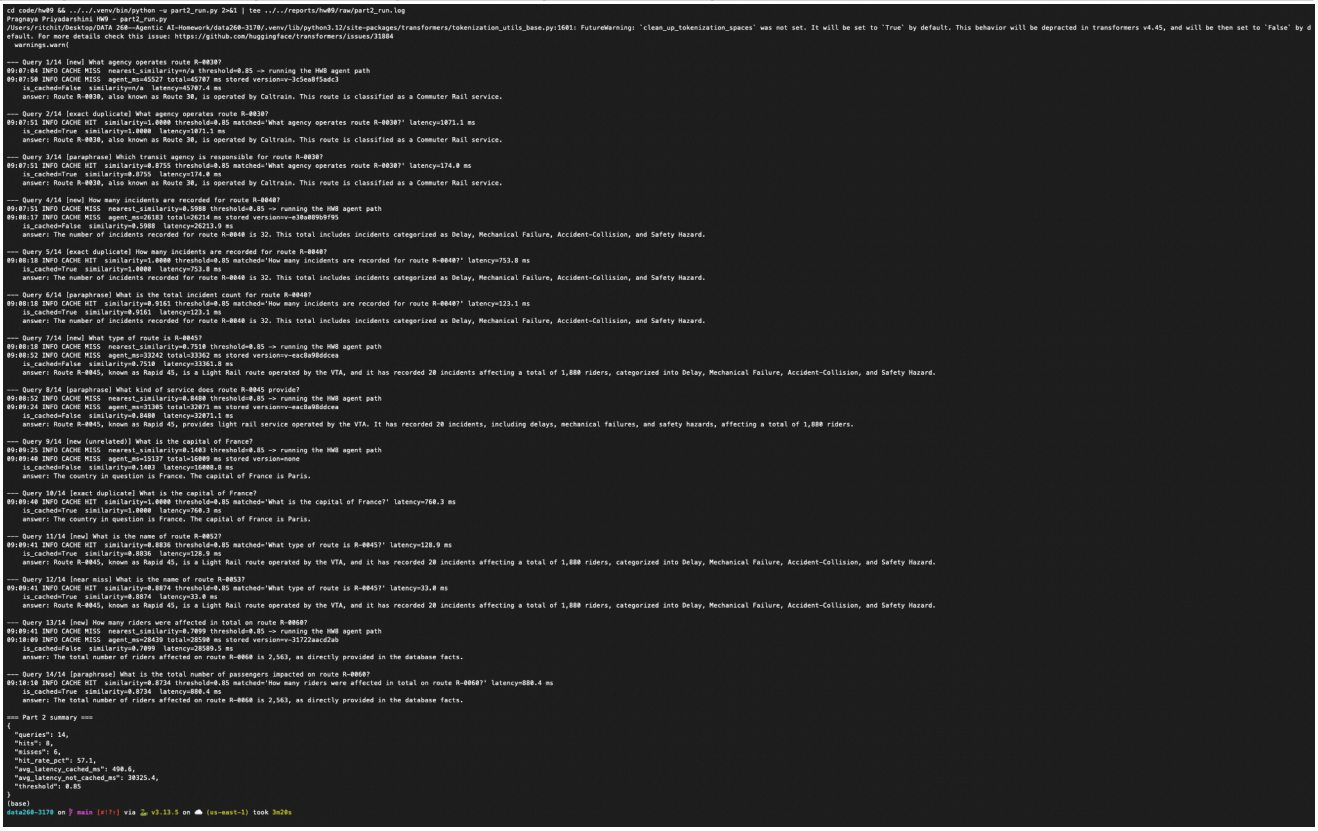
```
    def handle(self, query: str, threshold: float | None = None, validate_version: bool = True,
               agent: Callable[[str], dict[str, Any]] = run_agent) -> dict[str, Any]:
        """Ingress handler. hit -> stored answer, agent NOT run. miss -> HW8 agent path, then store."""
        t_start = time.perf_counter()
        look = self.lookup(query, threshold)
        near = look["neighbour"]
        stale_evicted = None
        if look["hit"] and validate_version:
            t_v = time.perf_counter()
            current = data_version(extract_entities(near["query"]))
            version_ms = (time.perf_counter() - t_v) * 1000
            if current != near["data_version"]:
                log.info("STALE  similarity=%.4f stored_version=%s current_version=%s -> evicting, treating as a miss",
                         near["similarity"], near["data_version"], current)
                self.invalidate(near["key"])
                stale_evicted = {"stored": near["data_version"], "current": current, "evicted_query": near["query"]}
                look["hit"] = False
            else:
                look["version_check_ms"] = version_ms
        if look["hit"]:
            total_ms = (time.perf_counter() - t_start) * 1000
            log.info("CACHE HIT  similarity=%.4f threshold=%.2f matched=%r latency=%.1f ms",
                     near["similarity"], look["threshold"], near["query"], total_ms)
            return {"query": query, "answer": near["answer"], "is_cached": True, "similarity": near["similarity"],
                    "matched_query": near["query"], "threshold": look["threshold"], "latency_ms": round(total_ms, 2),
                    "embed_ms": round(look["embed_ms"], 2), "search_ms": round(look["search_ms"], 2),
                    "version_check_ms": round(look.get("version_check_ms", 0.0), 2), "stale_evicted": None}
        sim = near["similarity"] if near else None
        log.info("CACHE MISS  nearest_similarity=%s threshold=%.2f -> running the HW8 agent path",
                 f"{sim:.4f}" if sim is not None else "n/a", look["threshold"])
        t_a = time.perf_counter()
        out = agent(query)
        agent_ms = (time.perf_counter() - t_a) * 1000
        self.store(query, out["answer"], out["data_version"], out["entities"], vec=look["vec"])
        total_ms = (time.perf_counter() - t_start) * 1000
        log.info("CACHE MISS  agent_ms=%.0f total=%.0f ms stored version=%s", agent_ms, total_ms, out["data_version"])
        return {"query": query, "answer": out["answer"], "is_cached": False, "similarity": sim,
                "matched_query": near["query"] if near else None, "threshold": look["threshold"],
                "latency_ms": round(total_ms, 2), "embed_ms": round(look["embed_ms"], 2),
                "search_ms": round(look["search_ms"], 2), "agent_ms": round(agent_ms, 2),
                "data_version": out["data_version"], "stale_evicted": stale_evicted}
```

Evidence - results at threshold 0.85

Hit rate 57.1% (8 of 14). Average latency: cached 490.6 ms, not cached 30,325.4 ms. These are measured values, not targets.

#	Kind	is_cached	Similarity	Latency (ms)	Query
1	new	False	n/a	45707.4	What agency operates route R-0030?
2	exact duplicate	True	1.0000	1071.1	What agency operates route R-0030?
3	paraphrase	True	0.8755	174.0	Which transit agency is responsible for route R-0030?
4	new	False	0.5988	26213.9	How many incidents are recorded for route R-0040?
5	exact duplicate	True	1.0000	753.8	How many incidents are recorded for route R-0040?
6	paraphrase	True	0.9161	123.1	What is the total incident count for route R-0040?
7	new	False	0.7510	33361.8	What type of route is R-0045?

#	Kind	is_cached	Similarity	Latency (ms)	Query
8	paraphrase	False	0.8480	32071.1	What kind of service does route R-0045 provide?
9	new (unrelated)	False	0.1403	16008.8	What is the capital of France?
10	exact duplicate	True	1.0000	760.3	What is the capital of France?
11	new	True	0.8836	128.9	What is the name of route R-0052?
12	near miss	True	0.8874	33.0	What is the name of route R-0053?
13	new	False	0.7099	28589.5	How many riders were affected in total on route R-0060?
14	paraphrase	True	0.8734	880.4	What is the total number of passengers impacted on route R-0060?



What the results show

- Hit latency is almost entirely the time to embed the query on the CPU. The Redis KNN search itself takes about 2 ms. Several hits took 0.75–1.1 s because the embedding ran while Ollama was still using the cores; hits not near an agent run took 30–175 ms.
- Query 11 was not a planned duplicate, yet it hit the cached answer for R-0045 (similarity 0.8836) and returned a wrong answer; query 12 hit the same entry. This is the Part 3 false-hit problem appearing early.
- Query 8 is a true paraphrase that scored 0.8480 and missed at 0.85.

Part 3 - Threshold Evaluation and Stale Answers

What I did

- Wrote 40 labeled queries before running any sweep and committed them (5eafe4c): 10 exact duplicates, 10 paraphrases, 10 superficially similar queries whose correct answer differs, 10 unrelated.
- Swept the threshold over 0.75 / 0.80 / 0.85 / 0.90 / 0.95, measured hit rate, false-hit rate, missed hits and latency, and priced each setting with a cost model.
- Showed a stale answer after a database update and fixed it with a data-version marker checked on every hit.

3.I Labeled test set

code/hw09/cache_cases.yaml. The near-miss labels were checked against MySQL before the sweep: every pair has a different correct answer. The file has not been edited since it was committed.

```yaml
version: 1
id4: 3170
prefix: s3170
embedding_model: sentence-transformers/all-MiniLM-L6-v2

seeds:
  - {id: S01, query: "What agency operates route R-0002?", key: {kind: agency, route: R-0002}}
  - {id: S02, query: "How many incidents are recorded for route R-0012?", key: {kind: incident_count, route: R-0012}}
  - {id: S03, query: "What type of route is R-0021?", key: {kind: route_type, route: R-0021}}
  - {id: S04, query: "How many Delay incidents are recorded for route R-0018?", key: {kind: category_count, route: R-0018, category: Delay}}
  - {id: S05, query: "What is the total number of riders affected on route R-0015?", key: {kind: riders_total, route: R-0015}}
  - {id: S06, query: "What is the name of route R-0033?", key: {kind: route_name, route: R-0033}}
  - {id: S07, query: "Which agency runs route R-0042?", key: {kind: agency, route: R-0042}}
  - {id: S08, query: "How many Safety Hazard incidents are recorded for route R-0050?", key: {kind: category_count, route: R-0050, category: Safety Hazard}}
  - {id: S09, query: "What type of route is R-0063?", key: {kind: route_type, route: R-0063}}
  - {id: S10, query: "How many incidents are recorded for route R-0071?", key: {kind: incident_count, route: R-0071}}
  - {id: S11, query: "What agency operates route R-0001?", key: {kind: agency, route: R-0001}}
  - {id: S12, query: "How many incidents are recorded for route R-0004?", key: {kind: incident_count, route: R-0004}}
  - {id: S13, query: "What type of route is R-0005?", key: {kind: route_type, route: R-0005}}
  - {id: S14, query: "How many Delay incidents are recorded for route R-0009?", key: {kind: category_count, route: R-0009, category: Delay}}
  - {id: S15, query: "What is the total number of riders affected on route R-0010?", key: {kind: riders_total, route: R-0010}}
  - {id: S16, query: "What is the name of route R-0011?", key: {kind: route_name, route: R-0011}}
  - {id: S17, query: "Which agency runs route R-0014?", key: {kind: agency, route: R-0014}}
  - {id: S18, query: "How many Safety Hazard incidents are recorded for route R-0019?", key: {kind: category_count, route: R-0019, category: Safety Hazard}}
  - {id: S19, query: "What type of route is R-0024?", key: {kind: route_type, route: R-0024}}
  - {id: S20, query: "How many incidents are recorded for route R-0027?", key: {kind: incident_count, route: R-0027}}
```

```python
def artifact_checks() -> None:
    import yaml

    def cases():
        p = HW9 / "cache_cases.yaml"
        d = yaml.safe_load(p.read_text())
        from collections import Counter
        g = Counter(c["group"] for c in d["cases"])
        exp = Counter(c["expect"] for c in d["cases"])
        return (len(d["cases"]) == 40 and g == {"exact_duplicate": 10, "paraphrase": 10, "near_miss": 10, "unrelated": 10}
                and exp == {"hit": 20, "miss": 20}), f"{len(d['cases'])} cases {dict(g)}"

    run("Part 3.1: cache_cases.yaml has 40 cases, 10 per group", cases)

    def committed():
        rel = "code/hw09/cache_cases.yaml"
        log = git("log", "--format=%H %ct", "--", rel).splitlines()
        if not log:
            return False, "not committed"
        mods = git("log", "--format=%H", "--diff-filter=M", "--", rel).splitlines()
        clean = git("status", "--porcelain", "--", rel) == ""
        sweep = RAW / "part3_sweep.json"
        first_commit_ts = int(log[-1].split()[1])
        before_sweep = (not sweep.exists()) or first_commit_ts <= int(sweep.stat().st_mtime)
        return clean and not mods and before_sweep, f"1 commit, never modified since, clean={clean}, committed before the sweep output={before_sweep}"

    run("Part 3.1: cache_cases.yaml was committed before the sweep and never relabeled", committed)
```

3.II Threshold sweep

Hit rate = hits / 40. False-hit rate = hits on the 20 should-miss cases / 20. Missed hits = should-hit cases that missed. Hit latency is measured (embed + search). Miss latency = measured lookup + the mean of 20 measured agent runs (28.9 s); the sweep did not re-run the agent for every miss.

Threshold	Hit rate	False-hit rate (of 20)	Missed hits (of 20)	Mean latency (ms)	Total cost ($)
0.75	75.0% (30/40)	50.0% (10/20)	0	7267.3	50.20
0.80	75.0% (30/40)	50.0% (10/20)	0	7255.0	50.20
0.85	67.5% (27/40)	50.0% (10/20)	3	9421.8	50.26
0.90	57.5% (23/40)	45.0% (9/20)	6	12310.9	45.34
0.95	47.5% (19/40)	35.0% (7/20)	8	15199.8	35.42

Hits per group (of 10 each):

Threshold	exact_duplicate	paraphrase	near_miss	unrelated
0.75	10	10	10	0
0.80	10	10	10	0
0.85	10	7	10	0
0.90	10	4	9	0
0.95	10	2	7	0



```
❯ make sweep-hw09
cd code/hw09 && ../../.venv/bin/python -u part3_sweep.py 2>&1 | tee ../../reports/hw09/raw/part3_sweep.log
Pragnaya Priyadarshini HW9 – part3_sweep.py
/Users/ritchit/Desktop/DATA 260—Agentic AI—Homework/data260-3170/.venv/lib/python3.12/site-packages/transformers/tokenization_utils_base.py:1601: FutureWarning: `clean_up_tokenization_spaces` was not set. It will be set to `True` by defa
ult. This behavior will be depracted in transformers v4.45, and will be then set to `False` by default. For more details check this issue: https://github.com/huggingface/transformers/issues/31884
  warnings.warn(
seed S01:  22430 ms  What agency operates route R-0002?  ->  The agency that operates route R-0002 is the VTA, as indicated by the database facts.
seed S02:  24252 ms  How many incidents are recorded for route R-0012?  ->  The number of incidents recorded for route R-0012 is 26. This total includes incidents categorized as Delay, Mechanical Failure, Accident-Collision, and Safety Ha
zard.
seed S03:  29631 ms  What type of route is R-0021?  ->  Route R-0021, known as Express 21, is a Bus route operated by Muni. It has recorded 23 incidents affecting 1,963 riders, including delays, mechanical failures, accidents, and safety
hazards.
seed S04:  23976 ms  How many Delay incidents are recorded for route R-0018?  ->  According to the database facts, there are 9 Delay incidents recorded for route R-0018.
seed S05:  35838 ms  What is the total number of riders affected on route R-0015?  ->  The total number of riders affected on route R-0015 is 2,861. This figure is directly provided in the database facts for route R-0015.
seed S06:  30723 ms  What is the name of route R-0033?  ->  The name of route R-0033 is Rapid 33. It is a Light Rail route operated by BART with a record of 20 incidents, affecting a total of 2,239 riders.
seed S07:  30172 ms  Which agency runs route R-0042?  ->  Route R-0042, known as the Express 42, is operated by the BART agency. It is a Subway route with a recorded history of incidents affecting a total of 2,584 riders.
seed S08:  32554 ms  How many Safety Hazard incidents are recorded for route R-0050?  ->  Route R-0050 has a total of 11 Safety Hazard incidents recorded according to the provided database facts.
seed S09:  30427 ms  What type of route is R-0063?  ->  Route R-0063, named Express 63, is a Bus route operated by SJ Transit. It is classified as an express bus route, as indicated by its name and type.
seed S10:  26022 ms  How many incidents are recorded for route R-0071?  ->  Route R-0071 has 21 incidents recorded in total. These incidents are categorized into Delay, Mechanical Failure, Accident-Collision, and Safety Hazard.
seed S11:  31046 ms  What agency operates route R-0001?  ->  The agency that operates route R-0001, known as Express 1, is AC Transit. This agency is responsible for managing and operating subway routes in the relevant area.
seed S12:  23877 ms  How many incidents are recorded for route R-0004?  ->  Route R-0004, named Express 4, has 24 incidents recorded according to the database facts.
seed S13:  34750 ms  What type of route is R-0005?  ->  Route R-0005, known as the SJ Transit Line 5, is a Light Rail route operated by SJ Transit. It has recorded 22 incidents affecting a total of 1,814 riders, categorized into delays, m
echanical failures, and safety hazards.
seed S14:  28036 ms  How many Delay incidents are recorded for route R-0009?  ->  Route R-0009, which is Line 9 operated by SamTrans, has recorded 6 Delay incidents according to the provided database facts.
seed S15:  33117 ms  What is the total number of riders affected on route R-0010?  ->  The total number of riders affected on route R-0010 is 1730. This figure is directly provided in the database facts for route R-0010.
seed S16:  29742 ms  What is the name of route R-0011?  ->  The name of route R-0011 is Route 11. It is a Subway route operated by BART with a recorded total of 21 incidents affecting 1,749 riders.
seed S17:  30147 ms  Which agency runs route R-0014?  ->  Route R-0014, known as Rapid 14, is operated by Caltrain, a commuter rail agency. The route is a Commuter Rail line managed by Caltrain, as indicated by the database facts.
seed S18:  23844 ms  How many Safety Hazard incidents are recorded for route R-0019?  ->  According to the database facts, there are 7 Safety Hazard incidents recorded for route R-0019.
seed S19:  29368 ms  What type of route is R-0024?  ->  Route R-0024, known as the SJ Transit Line 24, is a Subway route operated by SJ Transit. It has recorded incidents affecting a total of 2,604 riders, categorized primarily under Acci
dent-Collision, Safety Hazard, Mechanical Failure, and Delay.
seed S20:  27888 ms  How many incidents are recorded for route R-0027?  ->  The route R-0027 has 18 incidents recorded according to the database facts. These incidents are categorized as Delay=5, Mechanical Failure=7, Accident-Collision=3
, and Safety Hazard=3.

cache holds 20 entries; mean measured agent latency = 28892 ms

threshold  hit%   FH%   missedHits  meanLat(ms)  totalCost($)
0.75       75.0   50.0      0         7267.3      50.2
0.80       75.0   50.0      0         7255.0      50.2
0.85       67.5   50.0      3         9421.8      50.26
0.90       57.5   45.0      6        12310.9      45.34
0.95       47.5   35.0      8        15199.8      35.42

cheapest threshold at $5.0 per false hit: 0.95
(base)
```

3.III One false hit in full (threshold 0.85)

```
cached query   : 'How many incidents are recorded for route R-0071?'
incoming (C30) : 'How many incidents are recorded for route R-0072?'
similarity     : 0.9876
cached answer  : 'Route R-0071 has 21 incidents recorded in total. These incidents are
                  categorized into Delay, Mechanical Failure, Accident-Collision, and Safety
Hazard.'
correct answer : 28   (from s3170_rel)
```

Why it happened. The embedding model turns the question into one vector that mostly encodes what is being asked (a count of incidents on a route). “R-0071” and “R-0072” differ by one word-piece out of about 14, and identifiers carry almost no semantic weight, so the two sentences look nearly identical to the model. High similarity does not mean the same answer.



3.IV Cost model and shipped threshold

- **Cache-miss cost = $0.02.** One agent path is a planner call plus a writer call (sometimes one retry), roughly 2k tokens in and 300 out, which is 1–2 cents at hosted mid-tier pricing. It is paid on every miss. Locally it costs about 29 s of CPU instead.
- **False-hit cost = $5.00.** A rider or dispatcher gets a confident wrong fact and someone has to catch and fix it: about 10 minutes of support time at $30/h. That is 250 times the cost of a miss.
- **Total cost = misses × $0.02 + false hits × $5.** Removing one false hit is worth it only if it adds fewer than 250 extra misses.

Sensitivity of the total cost to the false-hit price ($):

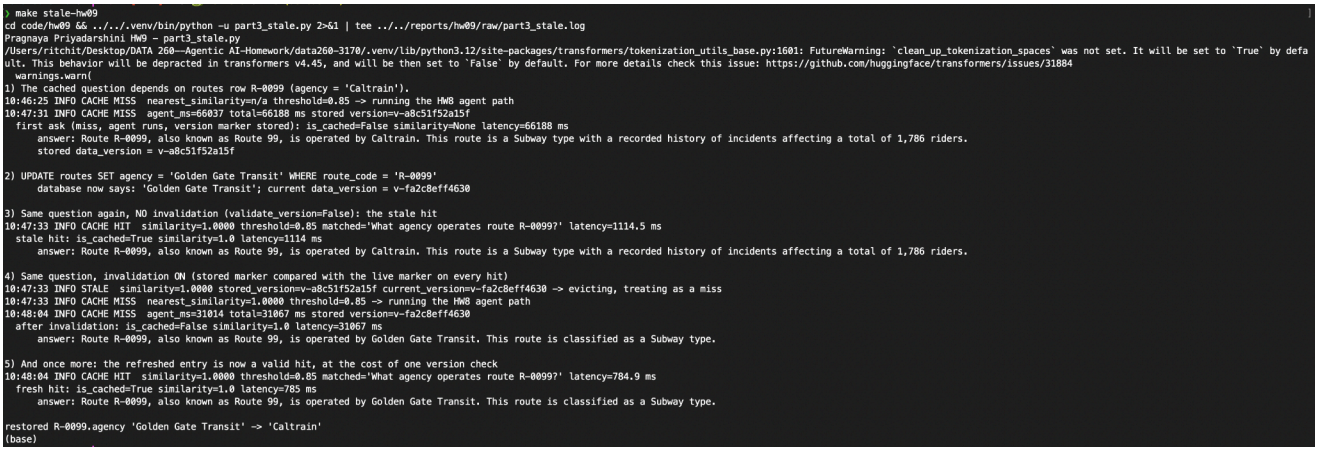
False-hit cost	0.75	0.80	0.85	0.90	0.95
$1	10.20	10.20	10.26	9.34	7.42
$5	50.20	50.20	50.26	45.34	35.42
$25	250.20	250.20	250.26	225.34	175.42

Shipped threshold: 0.95. It is the cheapest of the five at every false-hit price I tried. It is not good, only the least bad: 7 of the 20 should-miss cases still hit (all near-misses that differ in one route number or one category) and only 2 of 10 paraphrases hit. No threshold fixes this, because the confusable pairs (similarity 0.93–0.99) overlap the true paraphrases (0.84–0.98). An offline check on the same rows shows that requiring the same route code in both queries cuts false hits to 0 or 1 at every threshold. That guard is the next change, but it is not built and is not part of the table above.

3.V Stale answer and invalidation

The cached question “What agency operates route R-0099?” depends on the routes row for R-0099 (Caltrain). I updated that row's agency to Golden Gate Transit. With no invalidation the cache still answered Caltrain.

Strategy. Each cache entry stores a data-version marker: a hash of the route row's updated_at, the incident count and the latest incident updated_at. On every hit the handler recomputes the marker. If it differs, the entry is evicted (logged as STALE) and the agent runs again. The original agency was restored afterwards.



Trade-offs. Every hit now costs a database read (4.1 ms measured here, small next to a 30 s agent run, but it puts MySQL back on the hit path and does not scale to answers that depend on many rows). It only sees the rows named in the question, so an answer that silently depends on another table would still go stale. It also evicts on any update to those rows, even one that does not change the answer. Alternatives are a TTL (cheap, but stale until it expires) or event-driven deletion on write (no read cost, but every writer must remember to do it).

Part 4 - Self-Improving Two-Agent System

What I did

- Built an Answering Agent (qwen3:8b through the HW8 adapter) and an Evaluator Agent (deterministic, rule-based) in code/hw09/agents_si.py, with a lesson store and a loop that stops on PASS or after the second attempt, never a third.
- Reference dataset: 30 routes (R-0101 to R-0130) from s3170_rel plus a seeded fare history (synthetic, Random(3170)). Input files hold only {id, question}: training 12, validation 10, test 14.
- The Answering Agent sees only the route record its lookup returns, never the reference dataset. The Evaluator checks the answer against reference_dataset.json and returns {result: PASS | REVISE, explanation, feedback, failure_type}. On REVISE the feedback goes back to the Answering Agent together with its previous answer.

Datasets

```
reference_dataset.json  -> routes[0]: {route_code: "R-0101", route_name: "Line 101", agency:
"Caltrain",
                           route_type: "Commuter Rail", incident_count: 27, riders_total: 2186,
                           top_incident_category: "Delay", fare_history: [{fare_usd: 2.5,
effective: "2025-01-01"}, ...]}
training_data.json      -> {id: "tra_01", question: "What is the current fare on route R-0101?"}
...  (12)
validation_data.json    -> {id: "val_01", question: "What is the current fare for route
R-0111?"} ...  (10)
test_data.json          -> {id: "tes_01", question: "What is the current fare on route R-0121?"}
...   (14)
```

```
(base)
data260-3170 on  main [x!?↑] via  v3.13.5 on  (us-east-1) took 1m52s
❯ cd code/hw09 && ../../.venv/bin/python part4_run.py demo
Pragnaya Priyadarshini HW9 - part4_run.py demo
Question: Which agency operates route R-0110 and what type of route is it?
Controlled fault: attempt 1 is forced to 'Route R-0110 is a Commuter Rail route.'
    attempt 1: 'Route R-0110 is a Commuter Rail route.'
      evaluator: REVISE - The answer does not state the requested fact(s): agency.
      feedback: Your answer is missing: agency. Re-read the database record and state every requested fact explicitly.
    attempt 2: 'The agency that operates route R-0110 is AC Transit, and it is a Commuter Rail route.'
      evaluator: PASS - All requested facts match the reference dataset.
    => final PASS after 2 attempt(s), 15.03s
(base)
data260-3170/code/hw09 on  main [x!?↑] via  v3.13.5 on  (us-east-1) took 15s
❯
```

4.I Controlled revision case

```
question : Which agency operates route R-0110 and what type of route is it?
attempt 1: "Route R-0110 is a Commuter Rail route."
           REVISE  failure_type=missing_fact  feedback: Your answer is missing: agency. ...
attempt 2: "The agency that operates route R-0110 is AC Transit, and it is a Commuter Rail
route."
           PASS    attempts=2
```

```
    attempt 2: 'I do not have that information.'
      evaluator: PASS - The question is not covered by the reference data and the answer says so.
    => final PASS after 2 attempt(s), 14.55s
    ** pattern 'unsupported_answer' seen 2x -> created lesson_001

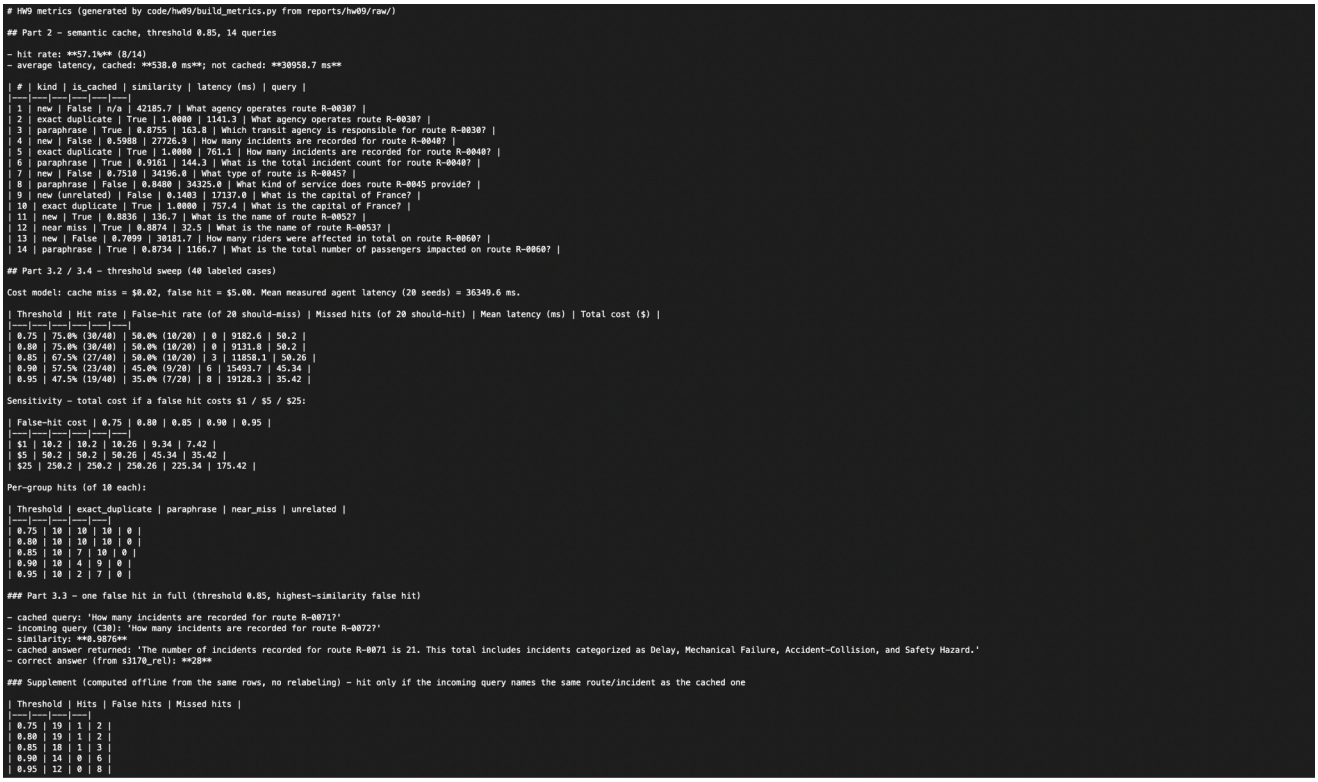
[tra_08] How many riders were affected in total on route R-0106, and which agency runs it?
    features=['multi_fact']
    attempt 1: 'A total of 1,559 riders were affected on route R-0106, which is run by AC Transit.'
      evaluator: PASS - All requested facts match the reference dataset.
    => final PASS after 1 attempt(s), 17.67s
--
    features=['no_record']
    attempt 1: 'I do not have that information in the transit database.'
      evaluator: PASS - The question is not covered by the reference data and the answer says so.
      lessons loaded: ['lesson_001']
    => final PASS after 1 attempt(s), 7.58s

[tra_12] What agency runs route R-0109?
    features=[]
    attempt 1: 'The agency that runs route R-0109 is AC Transit.'
      evaluator: PASS - All requested facts match the reference dataset.
(base)
data260-3170 on  main [x!?↑] via  v3.13.5 on  (us-east-1)
❯
```

4.II Lessons

During the 12-question training stream the pattern “answers an out-of-scope question from general knowledge” (failure type unsupported_answer) appeared twice (tra_03 and tra_07), so lesson_001 was created. It was loaded and applied at tra_11 (use_count 1), where the answer was a correct refusal on the first attempt. The stale-fare failure (tra_01) happened only once, so no lesson was created for it.

```
lesson_001  trigger=no_record  status=kept  use_count=1
  pattern   : The answer states a fact for a question that has no matching database record
(answered from general knowledge).
  correction: If the database says no record matched, do not answer from general knowledge.
Reply that you do not have
              that information in the transit database.
lesson_002  trigger=always     status=rejected  use_count=0
  correction: Only answer questions that contain a route code in the form R-NNNN. For every
other question, say you cannot answer it.
```



```
# HW9 metrics (generated by code/hw09/build_metrics.py from reports/hw09/raw/)

## Part 2 — semantic cache, threshold 0.85, 14 queries

- hit rate: **57.1%** (8/14)
- average latency, cached: **538.0 ms**; not cached: **30958.7 ms**

| # | kind | is_cached | similarity | latency (ms) | query |
|---|---|---|---|---|---|
| 1 | new | False | n/a | 42185.7 | What agency operates route R-0030? |
| 2 | exact duplicate | True | 1.0000 | 1141.3 | What agency operates route R-0030? |
| 3 | paraphrase | True | 0.8755 | 163.8 | Which transit agency is responsible for route R-0030? |
| 4 | new | False | 0.5988 | 27726.9 | How many incidents are recorded for route R-0040? |
| 5 | exact duplicate | True | 1.0000 | 761.1 | How many incidents are recorded for route R-0040? |
| 6 | paraphrase | True | 0.9161 | 144.3 | What is the total incident count for route R-0040? |
| 7 | new | False | 0.7510 | 34196.0 | What type of route is R-0045? |
| 8 | paraphrase | False | 0.8480 | 34325.0 | What kind of service does route R-0045 provide? |
| 9 | new (unrelated) | False | 0.1403 | 17137.0 | What is the capital of France? |
| 10 | exact duplicate | True | 1.0000 | 757.4 | What is the capital of France? |
| 11 | new | True | 0.8836 | 136.7 | What is the name of route R-0052? |
| 12 | near miss | True | 0.8874 | 32.5 | What is the name of route R-0053? |
| 13 | new | False | 0.7099 | 30181.7 | How many riders were affected in total on route R-0060? |
| 14 | paraphrase | True | 0.8734 | 1166.7 | What is the total number of passengers impacted on route R-0060? |

## Part 3.2 / 3.4 — threshold sweep (40 labeled cases)

Cost model: cache miss = $0.02, false hit = $5.00. Mean measured agent latency (20 seeds) = 36349.6 ms.

| Threshold | Hit rate | False-hit rate (of 20 should-miss) | Missed hits (of 20 should-hit) | Mean latency (ms) | Total cost ($) |
|---|---|---|---|---|---|
| 0.75 | 75.0% (30/40) | 50.0% (10/20) | 0 | 9182.6 | 50.2 |
| 0.80 | 75.0% (30/40) | 50.0% (10/20) | 0 | 9131.8 | 50.2 |
| 0.85 | 67.5% (27/40) | 50.0% (10/20) | 3 | 11858.1 | 50.26 |
| 0.90 | 57.5% (23/40) | 45.0% (9/20) | 6 | 15493.7 | 45.34 |
| 0.95 | 47.5% (19/40) | 35.0% (7/20) | 8 | 19128.3 | 35.42 |

Sensitivity — total cost if a false hit costs $1 / $5 / $25:

| False-hit cost | 0.75 | 0.80 | 0.85 | 0.90 | 0.95 |
|---|---|---|---|---|---|
| $1 | 10.2 | 10.2 | 10.26 | 9.34 | 7.42 |
| $5 | 50.2 | 50.2 | 50.26 | 45.34 | 35.42 |
| $25 | 250.2 | 250.2 | 250.26 | 225.34 | 175.42 |

Per-group hits (of 10 each):

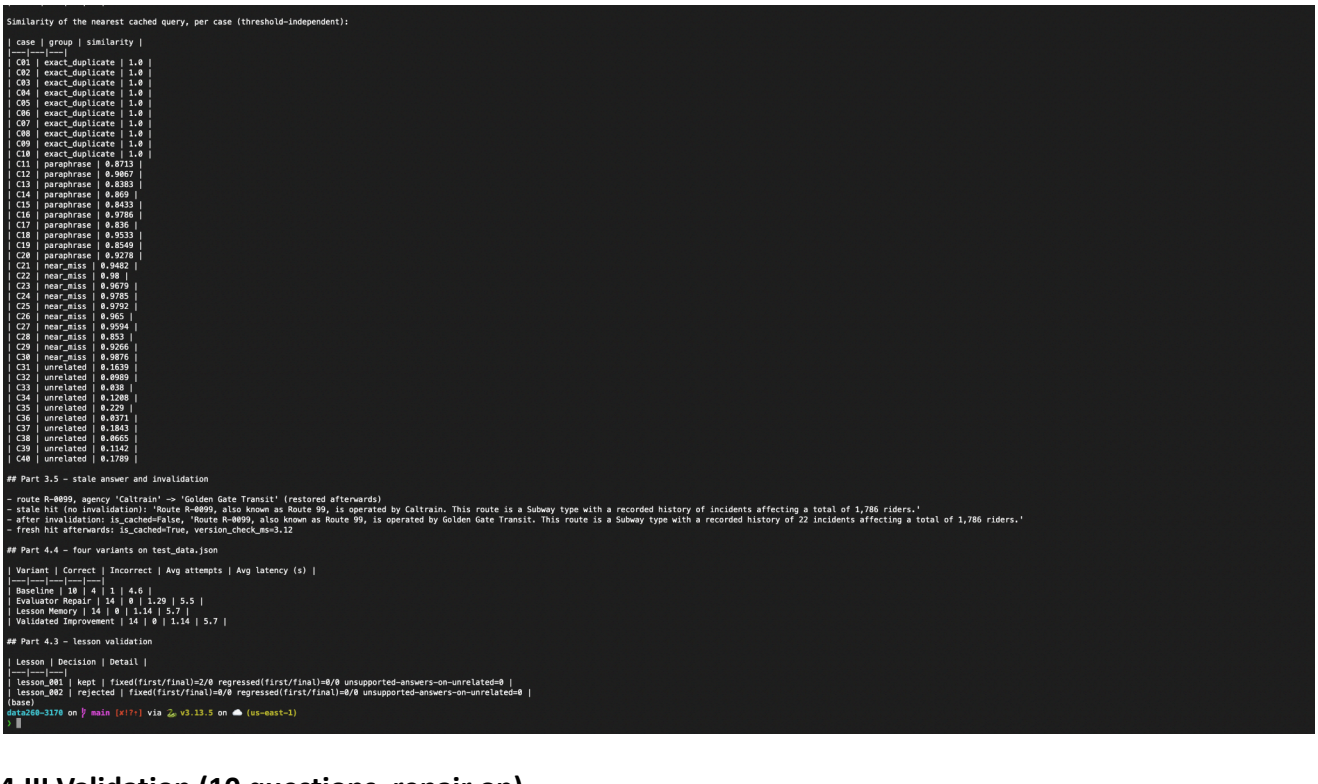
| Threshold | exact_duplicate | paraphrase | near_miss | unrelated |
|---|---|---|---|---|
| 0.75 | 10 | 10 | 10 | 0 |
| 0.80 | 10 | 10 | 10 | 0 |
| 0.85 | 10 | 7 | 10 | 0 |
| 0.90 | 10 | 4 | 9 | 0 |
| 0.95 | 10 | 2 | 7 | 0 |

### Part 3.3 — one false hit in full (threshold 0.85, highest-similarity false hit)

- cached query: 'How many incidents are recorded for route R-0071?'
- incoming query (C30): 'How many incidents are recorded for route R-0072?'
- similarity: **0.9876**
- cached answer returned: 'The number of incidents recorded for route R-0071 is 21. This total includes incidents categorized as Delay, Mechanical Failure, Accident-Collision, and Safety Hazard.'
- correct answer (from s3170_rel): **28**

### Supplement (computed offline from the same rows, no relabeling) — hit only if the incoming query names the same route/incident as the cached one

| Threshold | Hits | False hits | Missed hits |
|---|---|---|---|
| 0.75 | 19 | 1 | 2 |
| 0.80 | 19 | 1 | 2 |
| 0.85 | 18 | 1 | 3 |
| 0.90 | 14 | 0 | 6 |
| 0.95 | 12 | 0 | 8 |
```



```
Similarity of the nearest cached query, per case (threshold-independent):

| case | group | similarity |
|---|---|---|
| C01 | exact_duplicate | 1.0 |
| C02 | exact_duplicate | 1.0 |
| C03 | exact_duplicate | 1.0 |
| C04 | exact_duplicate | 1.0 |
| C05 | exact_duplicate | 1.0 |
| C06 | exact_duplicate | 1.0 |
| C07 | exact_duplicate | 1.0 |
| C08 | exact_duplicate | 1.0 |
| C09 | exact_duplicate | 1.0 |
| C10 | exact_duplicate | 1.0 |
| C11 | paraphrase | 0.8713 |
| C12 | paraphrase | 0.9067 |
| C13 | paraphrase | 0.8383 |
| C14 | paraphrase | 0.869 |
| C15 | paraphrase | 0.8433 |
| C16 | paraphrase | 0.9786 |
| C17 | paraphrase | 0.836 |
| C18 | paraphrase | 0.9533 |
| C19 | paraphrase | 0.8549 |
| C20 | paraphrase | 0.9278 |
| C21 | near_miss | 0.9482 |
| C22 | near_miss | 0.98 |
| C23 | near_miss | 0.9679 |
| C24 | near_miss | 0.9785 |
| C25 | near_miss | 0.9792 |
| C26 | near_miss | 0.965 |
| C27 | near_miss | 0.9594 |
| C28 | near_miss | 0.853 |
| C29 | near_miss | 0.9266 |
| C30 | near_miss | 0.9876 |
| C31 | unrelated | 0.1639 |
| C32 | unrelated | 0.0989 |
| C33 | unrelated | 0.038 |
| C34 | unrelated | 0.1208 |
| C35 | unrelated | 0.229 |
| C36 | unrelated | 0.0371 |
| C37 | unrelated | 0.1843 |
| C38 | unrelated | 0.0665 |
| C39 | unrelated | 0.1142 |
| C40 | unrelated | 0.1789 |

## Part 3.5 — stale answer and invalidation

- route R-0099, agency 'Caltrain' -> 'Golden Gate Transit' (restored afterwards)
- stale hit (no invalidation): 'Route R-0099, also known as Route 99, is operated by Caltrain. This route is a Subway type with a recorded history of incidents affecting a total of 1,786 riders.'
- after invalidation: is_cached=False, 'Route R-0099, also known as Route 99, is operated by Golden Gate Transit. This route is a Subway type with a recorded history of 22 incidents affecting a total of 1,786 riders.'
- fresh hit afterwards: is_cached=True, version_check_ms=3.12

## Part 4.4 — four variants on test_data.json

| Variant | Correct | Incorrect | Avg attempts | Avg latency (s) |
|---|---|---|---|---|
| Baseline | 10 | 4 | 1 | 4.6 |
| Evaluator Repair | 14 | 0 | 1.29 | 5.5 |
| Lesson Memory | 14 | 0 | 1.14 | 5.7 |
| Validated Improvement | 14 | 0 | 1.14 | 5.7 |

## Part 4.3 — lesson validation

| Lesson | Decision | Detail |
|---|---|---|
| lesson_001 | kept | fixed(first/final)=2/0 regressed(first/final)=0/0 unsupported-answers-on-unrelated=0 |
| lesson_002 | rejected | fixed(first/final)=0/0 regressed(first/final)=0/0 unsupported-answers-on-unrelated=0 |
(base)
data260-3170 on  main [✘!?↑] via  v3.13.5 on  (us-east-1)
❯
```

4.III Validation (10 questions, repair on)

Lesson	Decision	Detail
lesson_001	kept	Fixed 2 first-attempt failures (val_03, val_07); 0 regressions; no unsupported answers on out-of-scope questions

Lesson	Decision	Detail
lesson_002	rejected	Fixed nothing; 0 regressions

lesson_002 is a hand-written, over-broad rule (trigger: always) that I wrote to exercise the validation gate. I expected a regression, but the model mostly ignored it, so it was rejected for adding no benefit, not for harming accuracy. I am reporting that rather than claiming it caused a regression.

4.IV Results on test_data.json (14 questions, evaluator frozen)

Variant	Correct	Incorrect	Avg attempts	Avg latency (s)
Baseline	10	4	1.00	4.6
Evaluator Repair	14	0	1.29	5.5
Lesson Memory	14	0	1.14	5.7
Validated Improvement	14	0	1.14	5.7

This is run 2, on an otherwise idle machine. Run 1 gave the same correct/incorrect counts and attempt averages, but its latencies were 15.3, 7.6, 11.4 and 5.9 s because the machine was busy and the Baseline ran first. Latency differences in either run should not be read as an effect of the variants. Both runs are kept in reports/hw09/raw/.



What the numbers do and do not show

Repair lifts accuracy from 10 to 14 because every baseline failure (3 out-of-scope questions answered from general knowledge and 1 outdated fare) can be fixed from one round of feedback. Lessons do not add

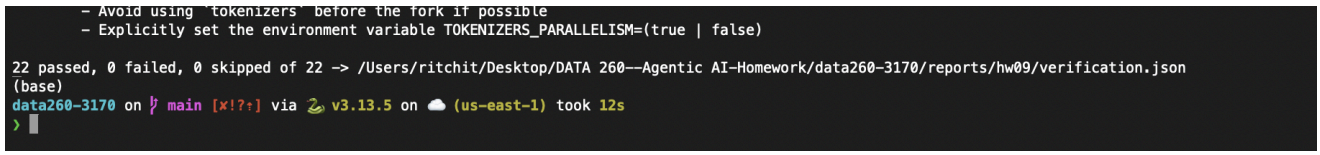
accuracy here because it is already 14 of 14. They cut average attempts from 1.29 to 1.14 by preventing 3 of the 4 first-attempt failures (the three out-of-scope questions). Lesson Memory and Validated Improvement tie because the only extra lesson Lesson Memory loads (lesson_002) was harmless on this test set. Four failing questions is a small sample.

Limitations

- **The evaluator's refusal check is a phrase list.** After the test run I saw it misjudge tes_12 (route R-9999). The Baseline's "There are no recorded incidents for route R-9999" passed because it contains "no record". The lesson variants' correct refusal "The transit database does not have information…" got REVISE because "does not have" is not in the list. I did not fix it, because the rules say not to modify the system after seeing the test set. It means the Baseline should score 9 of 14 and the lesson variants' average attempts would be 1.07, not 1.14. A stricter evaluator would check that a refusal contains no claim, for example with an LLM judge.
- The evaluator, the lesson triggers and the question-field keywords are hand-written rules matched to this question style; new phrasings would slip past them.
- The data is small: 12, 10 and 14 questions, one model, temperature 0 and one run each. Run 2 repeated run 1's correctness exactly, which is what temperature 0 predicts.

Self-check and repository

make verify-hw09 runs 22 checks with no LLM: the sweep table recomputed from the raw per-case rows; cache_cases.yaml committed before the sweep output existed and never modified; the 25-access counter run against real Redis and MySQL; a forced stale hit that the version check evicts; and Part 4 input files that contain only questions, with no test question in training or in any lesson (22 passed, 0 failed, 0 skipped of 22). Deliverables in reports/hw09/: RUN_LOG.txt, raw/, METRICS.md, AI_USE.md, verification.json and this report.



AI use statement

1. What I used an AI assistant for, and what I did myself

I used Claude Code (Sonnet 5.5) as a reference and guidance tool while completing HW9. I used it to clarify concepts, discuss possible implementation approaches, troubleshoot errors, and review parts of my work. I wrote and integrated the Redis view-counter method in cache_db.py, the RediSearch-based semantic cache in semantic_cache.py, the database-grounded Q&A agent in qa_agent.py, the threshold-sweep, stale-answer, Part 4, and verification scripts, and this final write-up.

I made the design decisions, selected the required constraints and labels, implemented the solution, ran the experiments, checked the outputs, and interpreted the results myself.

2. One AI suggestion that was unsuitable, and what I independently verified

- Homebrew Redis was already installed, but it did not include the required search module. I verified this using MODULE LIST, saw that only the vectorset module was available, and switched to the redis/redis-stack-server Docker image so that vector indexing could work.
- The first counter test printed “Synced views to database” inside the method, which caused the output order to differ from the handout. I identified the issue and moved the print statement to the test script.
- The first Part 4 test set included a question about “Route 128,” which was not present in reference_dataset.json. I checked the dataset, identified the problem, and changed the question before running the evaluation.
- I independently checked the ten different-answer near-miss labels against MySQL before running the sweep.
- I also verified the sweep results by recomputing the hit and false-hit counts from the raw results.

3. How I verified the implementation and results

I ran verify_hw09.py to verify the complete implementation. The verification checked that:

- The sweep results were recomputed from the raw per-case data.
- cache_cases.yaml was committed before the sweep output was generated and was not modified afterward.
- The view counter worked through 25 accesses using the real Redis and MySQL services.
- A stale cached answer was evicted after the corresponding route record was updated.
- The Part 4 input files contained only questions.
- No test question appeared in the training data or in any lesson.

4. What I changed and why it works

I corrected the Redis setup, fixed the counter test output order, replaced the invalid Part 4 question, and verified the results using the actual Redis and MySQL services. These changes ensure that the implementation follows the assignment requirements and that the reported results are based on reproducible checks.

The following limitations are stated clearly:

- The sweep’s miss latency is the mean of 20 measured agent runs, rather than a new agent run for every individual miss.
- The near-miss false hits are not eliminated simply by increasing the similarity threshold.
- The broad candidate lesson in Part 4.III was intentionally written by hand to test the filtering gate.

References

- Redis Search and Query (vector fields, HNSW, KNN queries): https://redis.io/docs/latest/develop/interact/search-and-query/
- redis-py client: https://redis-py.readthedocs.io/
- Sentence-Transformers: https://www.sbert.net/ and the all-MiniLM-L6-v2 model card: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2